Unit No. 1

Overview and System Calls of an Operating System

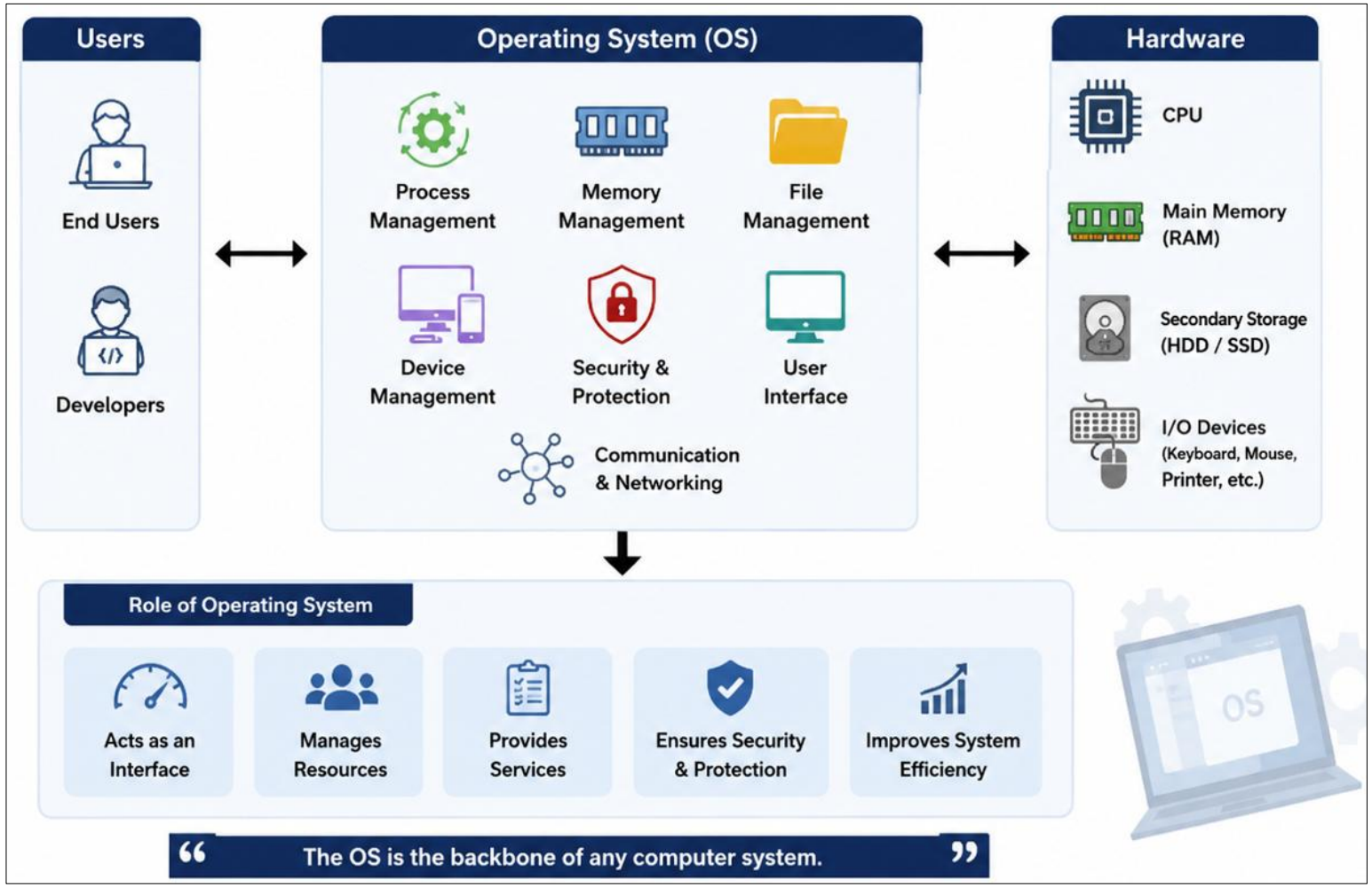
Course Outcome 1

Use an operating system effectively and understand the concept of system calls

Unit No. 1

- What is an Operating System? Functions of an OS;
- Types: Batch Operating System, Multiprogrammed Operating System, time-sharing operating system, Multiprocessor Operating System, Distributed Operating System, Real-Time Operating System;
- Different services of Operating Systems,
- System Calls ~ Concept, Types of system calls.

Unit No. 1
Overview and System Calls of an Operating System



An **Operating System** is software that controls and manages all the resources of a computer, such as the CPU, memory, storage, and input/output devices, while providing a platform for users and applications to interact with the hardware.

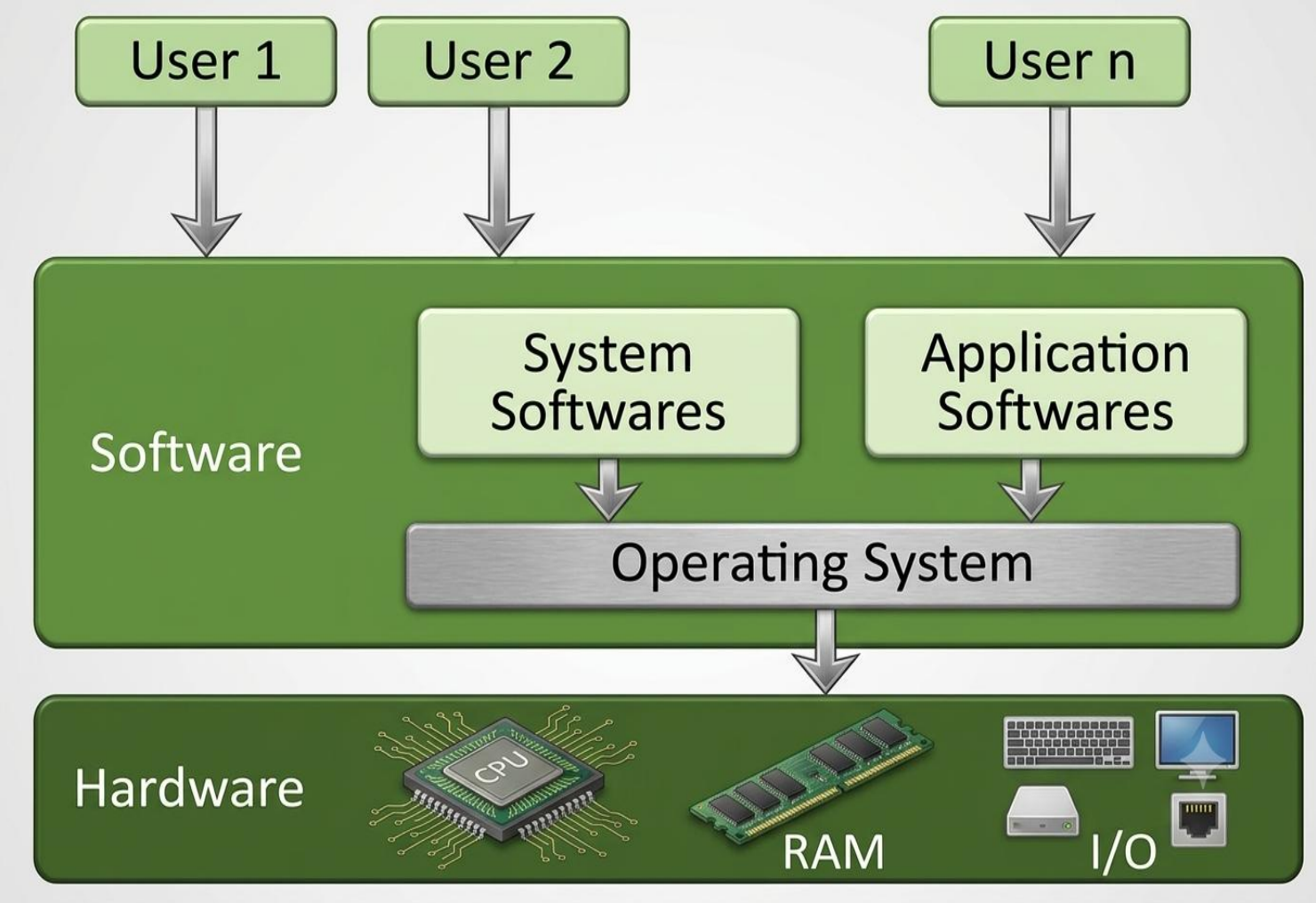
What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner
- **Example Operating Systems:**
  - Microsoft Windows, Linux, macOS, Android, iOS, UNIX

Key Points:-

- Acts as an interface between **the user** and **Hardware**
- Manages **CPU, Memory, Files, and Devices**
- Controls the execution of programs
- Provides security and resource allocation
- Enables users to interact with the computer efficiently

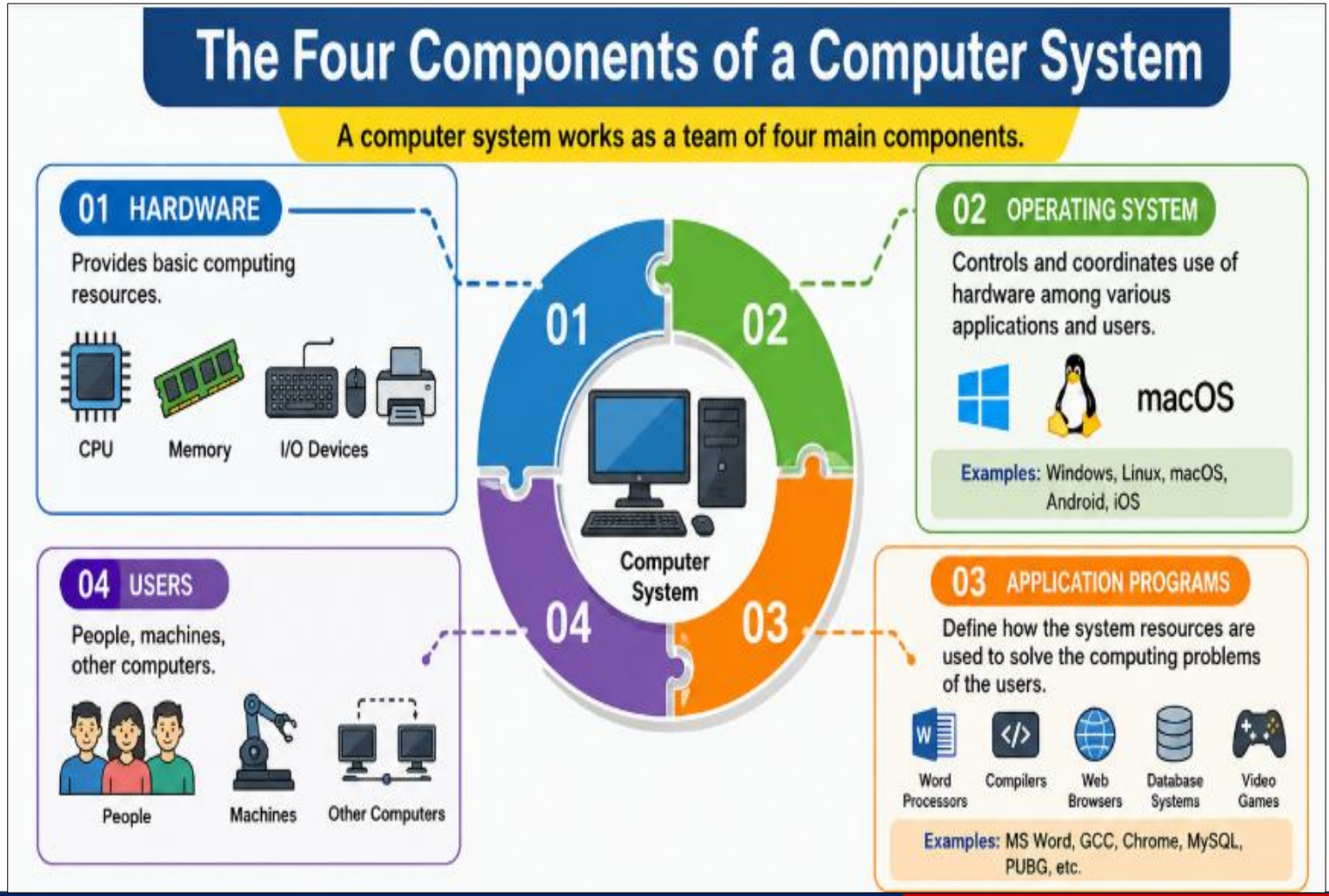
Computer System Components



Computer System Components

A computer system can be divided into four components:

- Hardware – provides basic computing resources
  - CPU, memory, I/O devices
- Operating system
  - Controls and coordinates use of hardware among various applications and users
- Application programs – define how the system resources are used to solve the computing problems of the users
  - Word processors, compilers, web browsers, database systems, video games
- Users
  - People, machines, other computers



What is an Operating System?

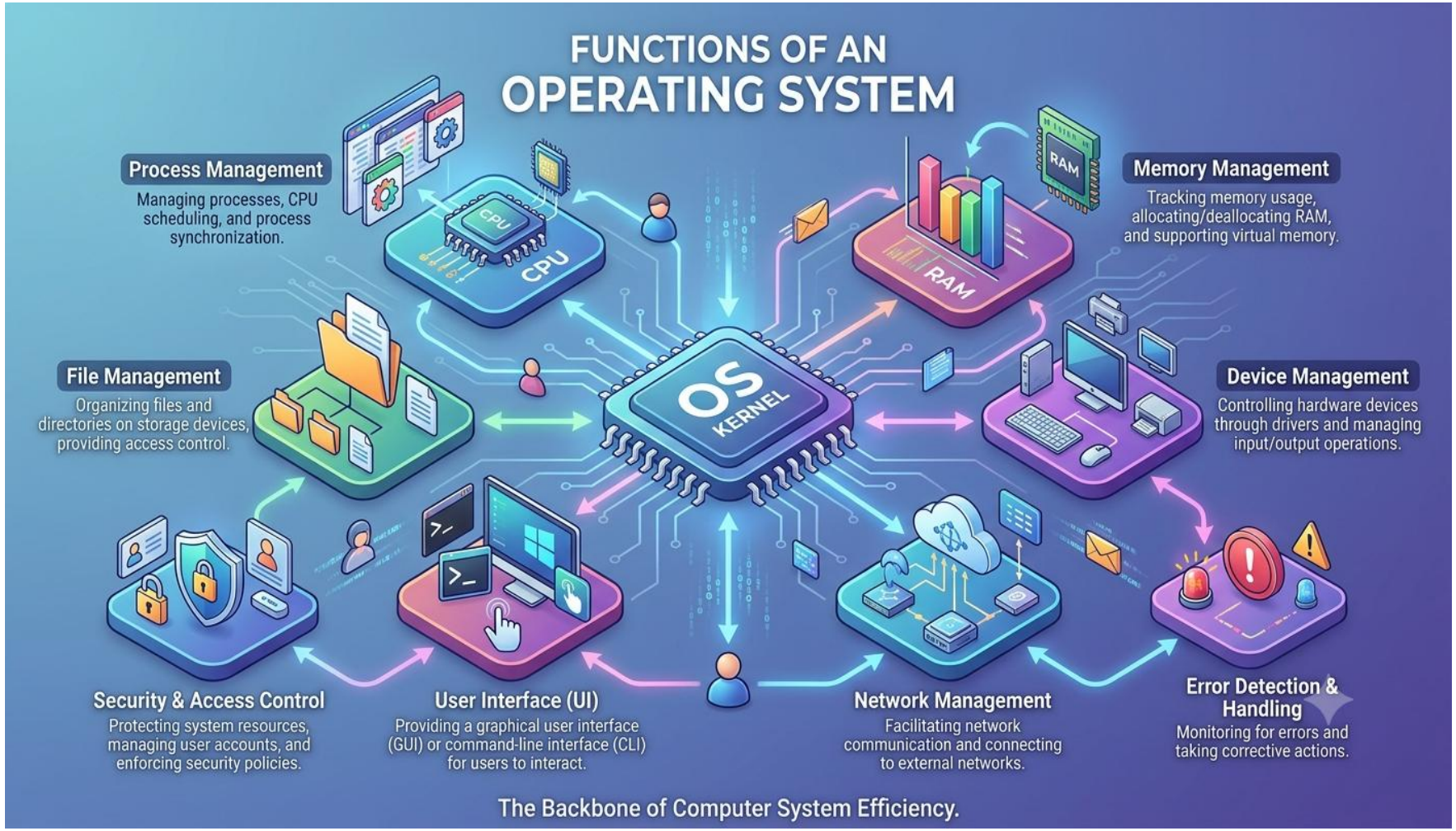
- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner
- **Example Operating Systems:**
  - Microsoft Windows, Linux, macOS, Android, iOS, UNIX

Key Points:-

- Acts as an interface between **the user** and **Hardware**
- Manages **CPU, Memory, Files, and Devices**
- Controls the execution of programs
- Provides security and resource allocation
- Enables users to interact with the computer efficiently

Functions of an OS:

- An operating system performs various functions to manage computer resources effectively and provide services to users and applications



Functions of an OS:

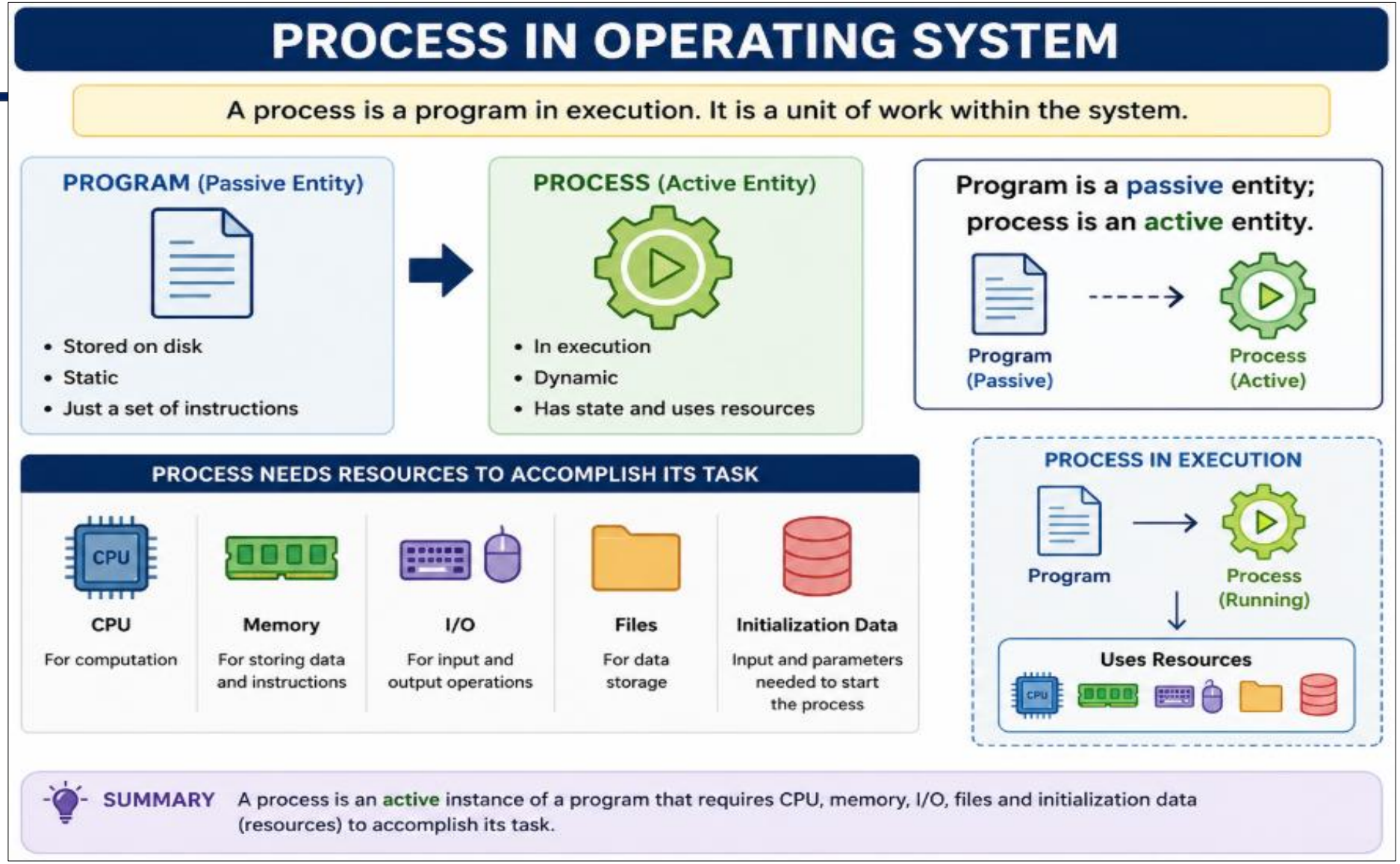
1. Process Management

- **Function:** Manages processes, including CPU scheduling, creation, and synchronization.
- The OS ensures each process receives a fair amount of CPU time.



Functions of an OS:

1. Process Management



Functions of an OS:

1. Process Management

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

A *deadlock* in an OS is a situation in which more than one process is blocked because it is holding a resource and also requires a resource that is acquired by some other process.

Functions of an OS:

2. Memory Management

- **Function:** Tracks memory usage (RAM) and allocates/deallocates memory space to different processes.
- It also manages virtual memory if physical RAM is full.



Functions of an OS:

2. Memory Management

- To execute a program, all (or part) of the instructions must be in memory.
- All (or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when
  - Optimizing CPU utilization and computer response to users

Memory management activities

- Keeping track of which parts of memory are currently being used and by whom
- Deciding which processes (or parts thereof) and data to move into and out of memory
- Allocating and deallocating memory space as needed

Functions of an OS:

3. Device Management

- **Function:** Controls hardware devices through their respective drivers.
- It manages all input and output operations between the system and peripherals.



Functions of an OS:

4. File Management

- **Function:** Manages the organization, storage, retrieval, and sharing of files on storage devices.
- It maintains directory structures and provides access control.



Functions of an OS:

4. Resource Management- File System Management

- OS provides a uniform, logical view of information storage
  - Abstracts physical properties to logical storage unit - **file**
- File-System management
  - Files usually organized into directories
  - Access control on most systems to determine who can access what
  - **File management activities**
    - Creating and deleting files and directories
    - Primitives to manipulate files and directories
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media

Functions of an OS:

5. Security & Access Control

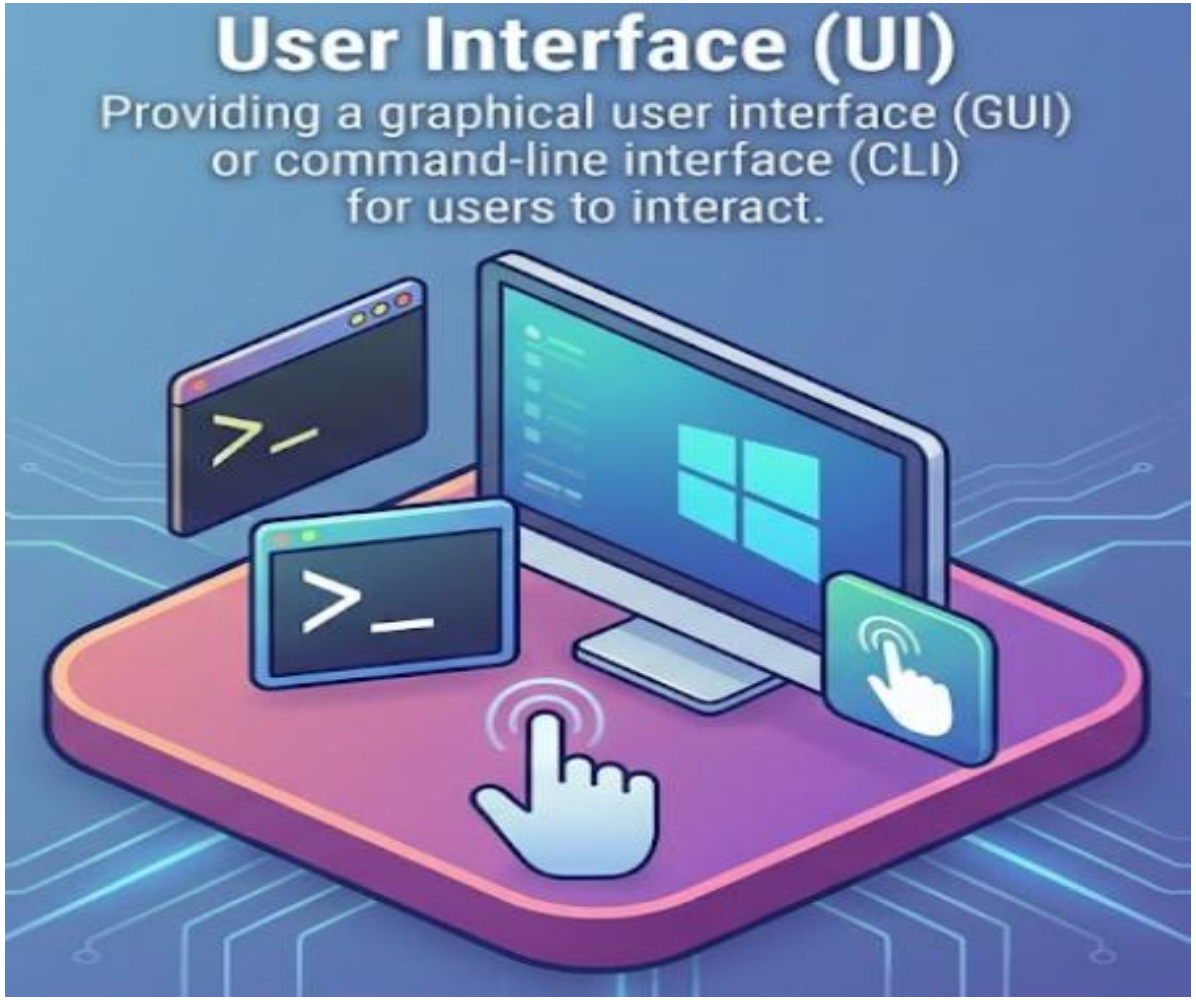
- **Function:** Protects the system from unauthorized access and malicious software.
- It manages user authentication and enforces security policies to safeguard data.



Functions of an OS:

6. User Interface (UI)

- **Function:** Provides the means for a user to interact with the computer.
- This can be a Graphical User Interface (GUI), with windows and icons, or a Command-Line Interface (CLI), using text commands.



Functions of an OS:

7. Network Management

- **Function:** Manages network resources and facilitates communication between the computer and external networks.
- This includes handling data packets and maintaining connections.



Functions of an OS:

8. Error Detection & Handling

- **Function:** Constantly monitors the system for errors or malfunctions.
- When an error is detected, the OS takes corrective actions to prevent data loss or system failure.



Operations of OS :-

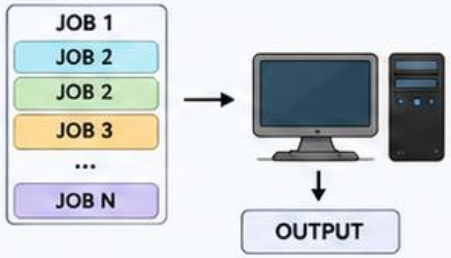
- Transferring input from the input device to memory.
- Transferring input from memory to the CPU.
- Transferring data from the CPU to the output device.
- Copy or move data from one storage device to another.
- It manages the programs running on the system
- Display appropriate error messages in case of software or hardware problems.
- Maintain details of files & directories.
- To provide security to users' data.
- Protection of storage devices from overwriting.
- It manages the hardware and software resources of the system.
- It provides a stable, consistent way for applications to deal with the hardware without knowing all the details of the hardware.



TYPES OF OPERATING SYSTEM

Operating Systems are the backbone of computers.
They manage resources and provide services for users and applications.

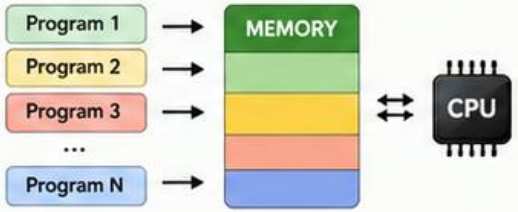
1 Batch Operating System



- Similar jobs are collected and executed in batches.
- No user interaction during execution.

Example: Early IBM Systems

2 Multiprogramming Operating System



- Multiple programs are kept in memory.
- CPU switches among them to maximize utilization.

Example: OS/360, UNIX

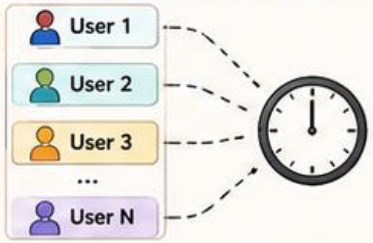
3 Multitasking Operating System



- Many tasks run simultaneously.
- CPU switches between tasks rapidly.

Example: Windows, macOS, Linux

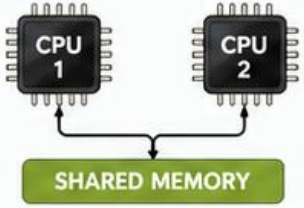
4 Time-sharing Operating System



- CPU time is divided into small time slots.
- Many users get interactive service simultaneously.

Example: UNIX, Windows, Linux

5 Multiprocessor OS / Tightly Coupled / Parallel System



- Uses two or more CPUs working together.
- Improves speed, reliability and parallel processing.

Example: Linux, Windows Server

6 Distributed Operating System / Loosely Coupled System



- Manages a group of independent computers.
- Makes them appear as a single system.

Example: Amoeba, Plan 9

7 Real-time Operating System



- Provides immediate response within strict time limits.
- Used in critical and time-sensitive applications.

Example: VxWorks, QNX, RTLinux

8 Mobile Operating System



- Designed for smartphones, tablets and mobile devices.
- Optimized for touch, battery and connectivity.

Example: Android, iOS, Windows Phone

Remember: Every Operating System is designed for specific needs. Understanding their types helps us choose the right system for the right task!

Types of OS:

- Batch Operating System
- Multiprogramming Operating System
- Multitasking Operating System
- Time-sharing Operating System
- Multiprocessor OS/Tightly couples/Parallel System
- Distributed Operating System/Loosely coupled System
- Real-time Operating System
- Mobile Operating System

Types of OS: 1. Batch Operating System

- A batch stands for a sequence of user jobs.
- The programmers leave their programs with the operator; the operator then sorts the programs with similar requirements into batches, & submit it to the **batch processing system (BPS)** & then those jobs are executed automatically without user interaction.
- The users of a batch operating system do not interact with the computer directly.
- The jobs were recognized with special markers that indicate the start & end of jobs.
- BPS consists of a batch monitor or supervisor was the major component which permanently placed in part of memory, & other part of memory is used to process the user's job

BATCH OPERATING SYSTEM

A batch stands for a sequence of user jobs. The programmers leave their programs with the operator; the operator then sorts the programs with similar requirements into batches, & submit it to the batch processing system (BPS) & then those jobs are executed automatically without user interaction.

1. USERS / PROGRAMMERS



Programmers write programs and submit them to the operator.

2. OPERATOR



Operator collects programs and sorts them with similar requirements into batches.

3. BATCH FORMATION

BATCH

- JOB 1
- JOB 2
- JOB 3
- ...
- JOB N

Jobs with similar requirements are grouped together to form a batch.

4. BATCH PROCESSING SYSTEM (BPS)



The batch is submitted to BPS and jobs are executed automatically without user interaction.

5. OUTPUT



Output is produced after execution of the entire batch.



The users of a batch operating system do not interact with the computer directly.

JOB STRUCTURE WITH SPECIAL MARKERS

The jobs were recognized with special markers that indicate the start & end of jobs.

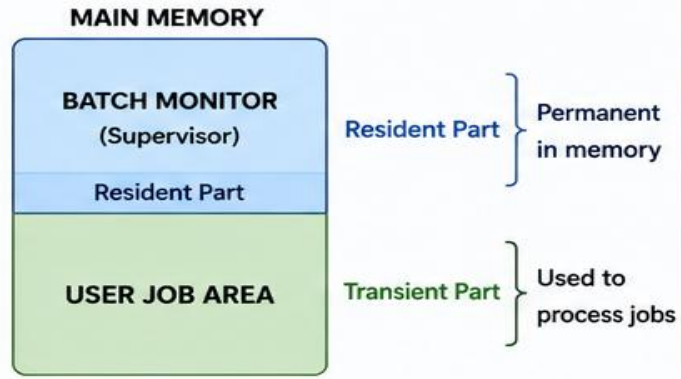


- $AMJ – Start of Job
- $DTA – End of Job

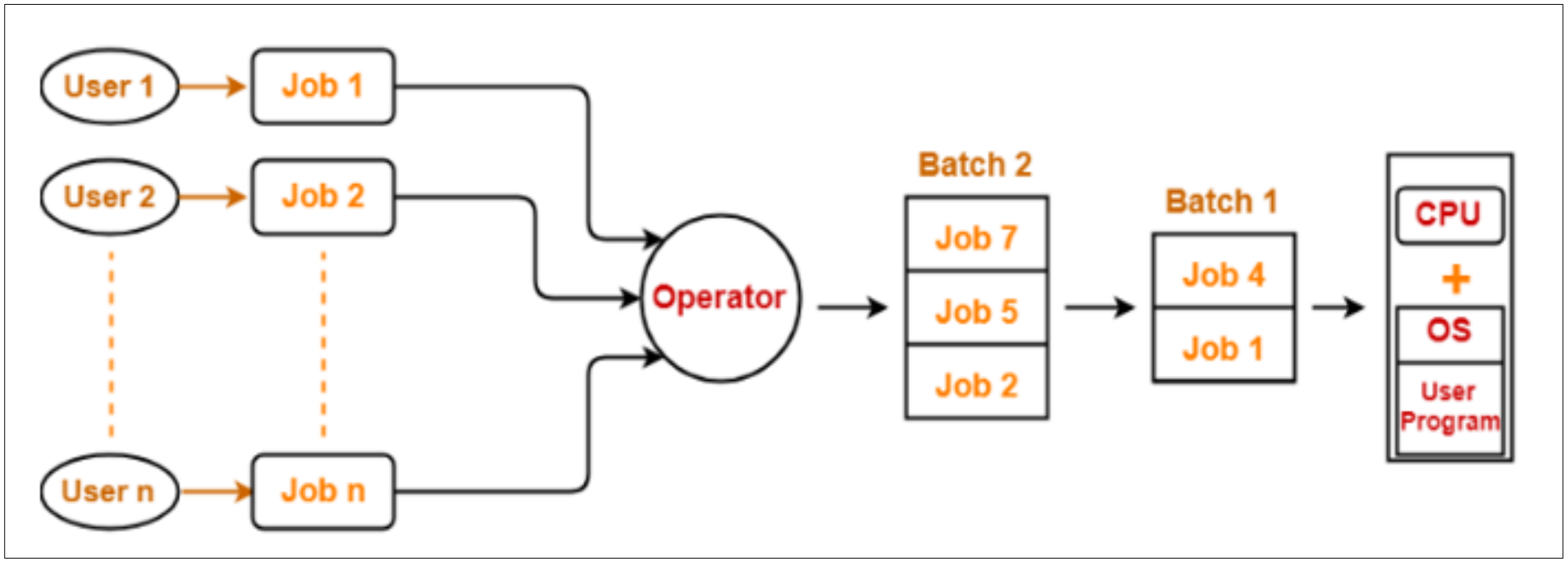
These markers help the system identify where a job begins and ends.

BPS (BATCH PROCESSING SYSTEM)

BPS consists of a batch monitor or supervisor which is permanently placed in part of memory, & other part of memory is used to process the user's job.



Types of OS: 1. Batch Operating System



Types of OS: 1. Batch Operating System- Functions

- **Batch Monitor**
  - Batch monitor keeps control of the processing environment
- **Scheduling:**
  - This activity determines which service request should be handled in the next turn.
  - BPS uses the First Come First Serve (FCFS) algorithm for scheduling.
- **Memory management:-**
  - During BPS operation, memory is divided into two parts
    - System working area 2. User working area.
  - Partitioning of memory is done by the OS.
  - Some part of the monitor is permanently stored in memory, known as the resident area of the monitor.
- **Sharing & protection :**
  - By dividing the memory into two parts, protection is very well provided.

Types of OS: 1. Batch Operating System

- **Advantages of Batch OS:**
- Repeated jobs are done fast in batch systems without user interaction.
- You don't need special hardware and system support to input data in batch systems.
- Best for large organizations
- Sharing of a batch system by multiple users.
- The idle time of a batch system is very low.
- You can assign a specific time for the batch jobs, so when the computer is idle, it starts processing the batch jobs, i.e., at night or any free time.
- The batch system can manage large repeated work easily.
- Programmer attention is reduced.

Disadvantages of Batch OS:

- The monitor remains in memory it results in a waste of memory wastage
- It is difficult to debug batch systems. Lack of protection
- Batch systems are sometimes costly.

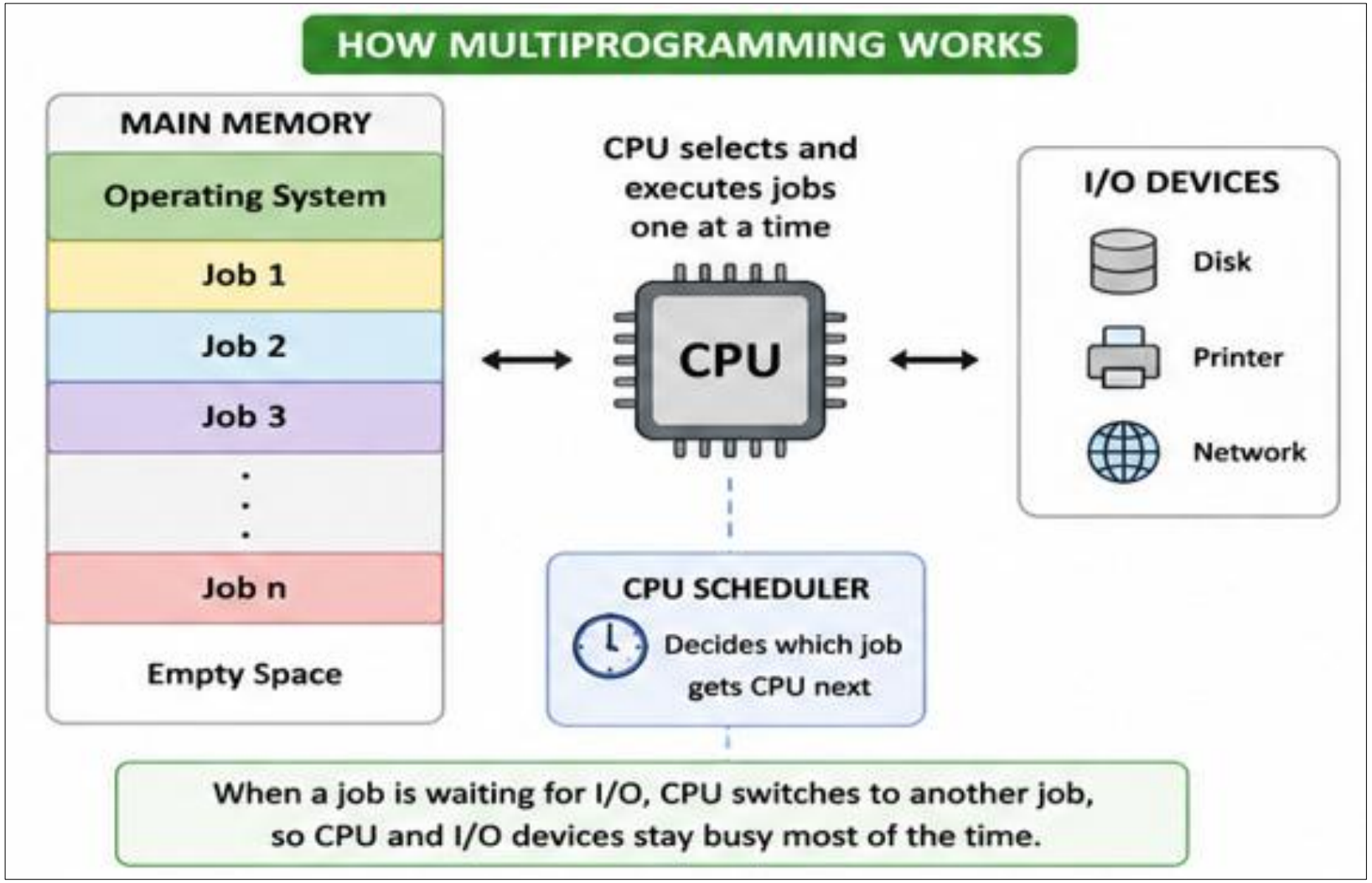
Types of OS: 2. Multiprogrammed Operating System

- The process of executing multiple programs simultaneously is known as **multiprogramming.**
- Uni-programming cannot keep the CPU & I/O devices busy at all times.
- Using multiprogramming, we can increase the utilization of the CPU & I/O devices.
- Multiprogramming OS keeps several jobs in memory at a time.
- E.g., a computer running Excel and Firefox browser simultaneously



Multiprogramming OS keeps several jobs in memory at a time and allows the CPU to switch between them. This increases the utilization of CPU and I/O devices.

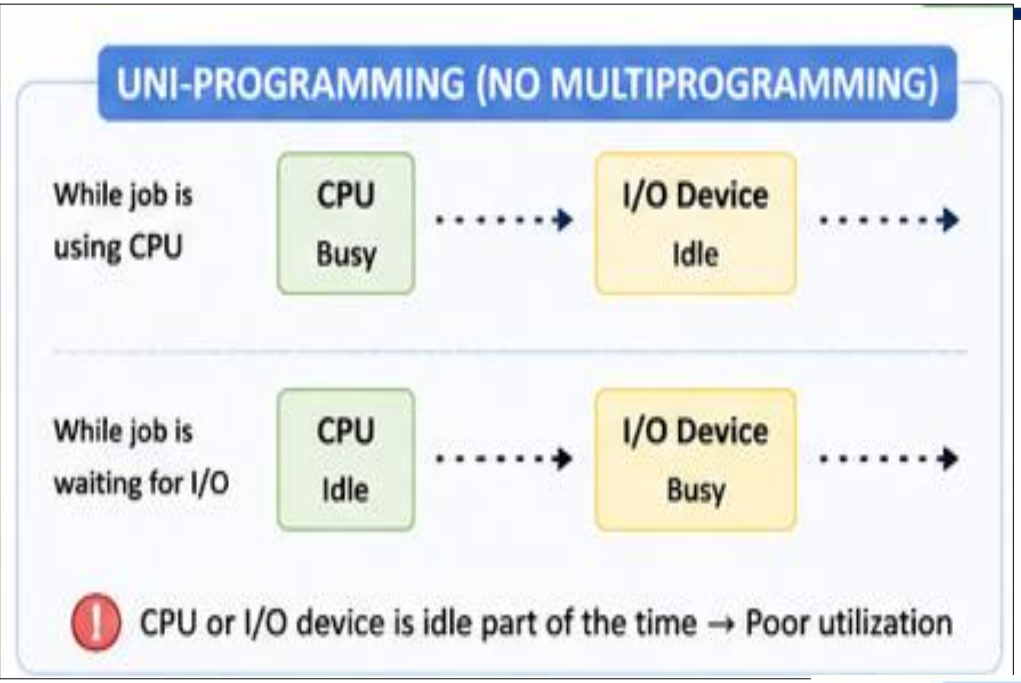
Types of OS: 2. Multiprogrammed Operating System

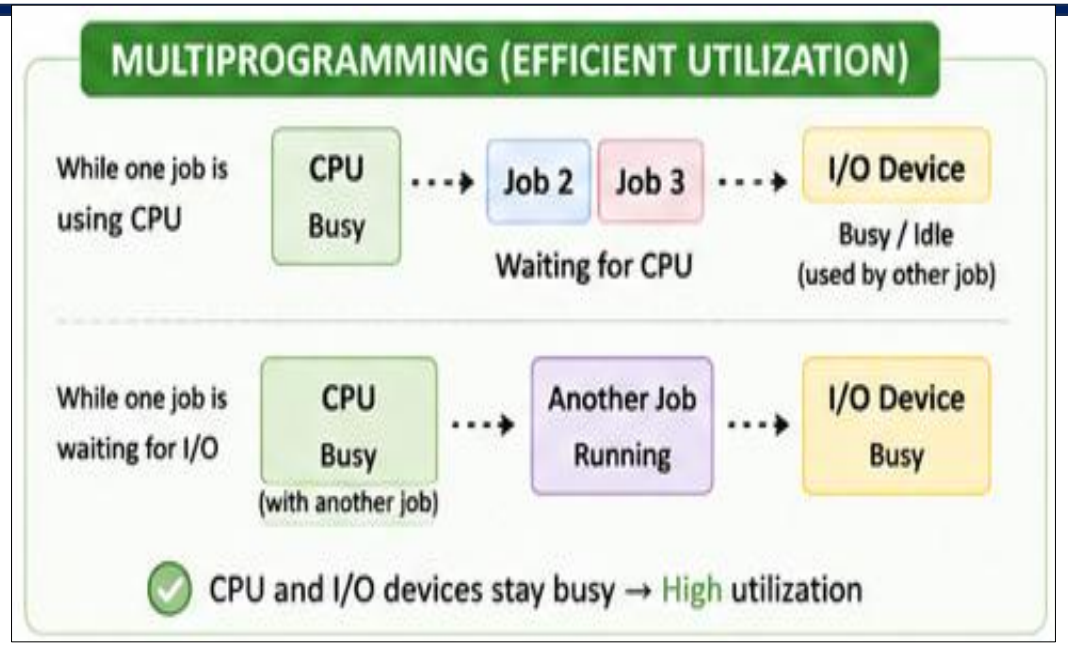


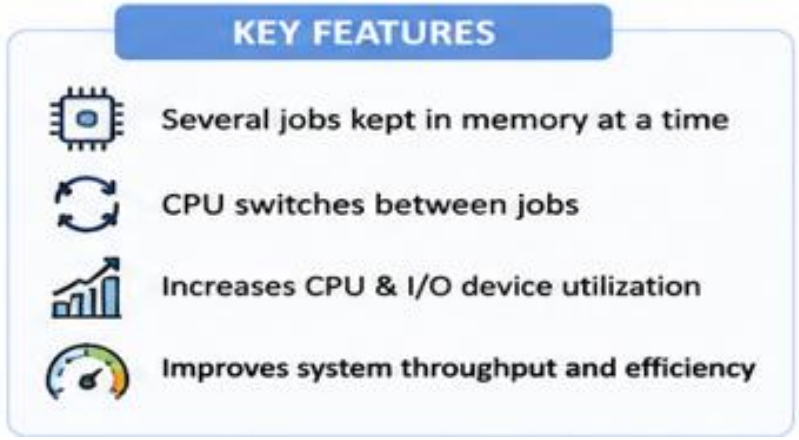
WHY MULTIPROGRAMMING?

- Uni-programming cannot keep the CPU & I/O devices busy at all times.
- Multiprogramming increases the utilization of the CPU & I/O devices.
- The process of executing multiple programs simultaneously is called multiprogramming.
- Several jobs are kept in memory and executed by the OS.

Types of OS: 2. Multiprogrammed Operating System







Types of OS: 2. Multiprogrammed Operating System

Advantages and Disadvantages of Multiprogramming OS

ADVANTAGES

Reduced Setup Time
Jobs are loaded together, reducing idle time.

Improved Efficiency
CPU and I/O devices are used more effectively.

Reduced Waiting Time
While one job waits for I/O, another uses the CPU.

Higher CPU Utilization
CPU stays busy, increasing system throughput.

DISADVANTAGES

Requires Memory Management
OS must manage memory for multiple programs.

Needs Scheduling Algorithm
CPU scheduler must choose the next job efficiently.

Higher Maintenance Cost
More complex OS leads to higher maintenance cost.

Complex OS Design
Implementation and management are more complex.

PROGRAM, PROCESS AND TASK

Program is a set of instructions. When a program is executed, it becomes a **process**.
A process performs different activities which are called **tasks**.

1. PROGRAM

A program is a set of instructions stored on the disk.



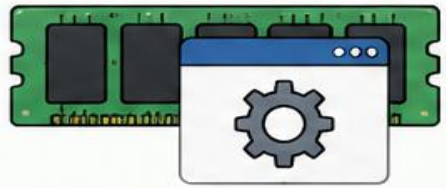
Example

Notepad.exe (Stored on Disk)

- Passive entity
- Does not consume CPU until executed

2. PROCESS

When a program is executed and loaded into memory, it becomes a process.



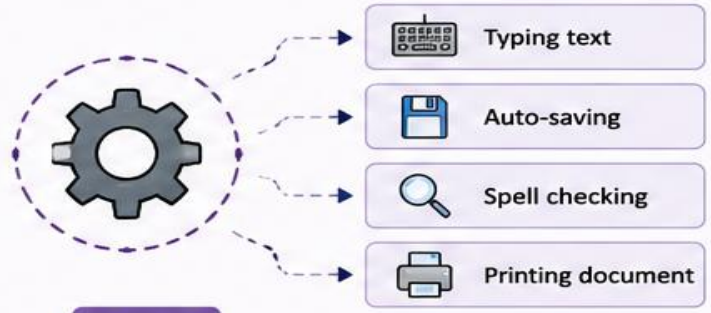
Example

Notepad.exe is running in the memory (Process)

- Active entity
- Uses CPU, memory and I/O resources

3. TASK

A process performs many activities.
Each activity is called a **task**.



Example

Typing, saving, spell checking, printing etc. are **tasks of Notepad process.**

- Unit of work within a process
- Smallest schedulable work unit (Thread in many OS)

RELATIONSHIP

PROGRAM
(Stored on Disk)



Execute

PROCESS
(Running in Memory)



Performs various activities

TASKS
(Individual Operations)

EXAMPLE 1 – Microsoft Word

Program	Process	Tasks
 WINWORD.EXE (Stored on Disk)	Execute →  WINWORD.EXE (Running in Memory)	Performs Tasks Tasks: • Typing • Auto-saving • Spell checking • Printing

EXAMPLE 2 – Google Chrome

Program	Process	Tasks
 chrome.exe (Stored on Disk)	Execute →  chrome.exe (Running in Memory)	Performs Tasks Tasks: • Loading webpage • Playing video • Downloading file • Checking updates

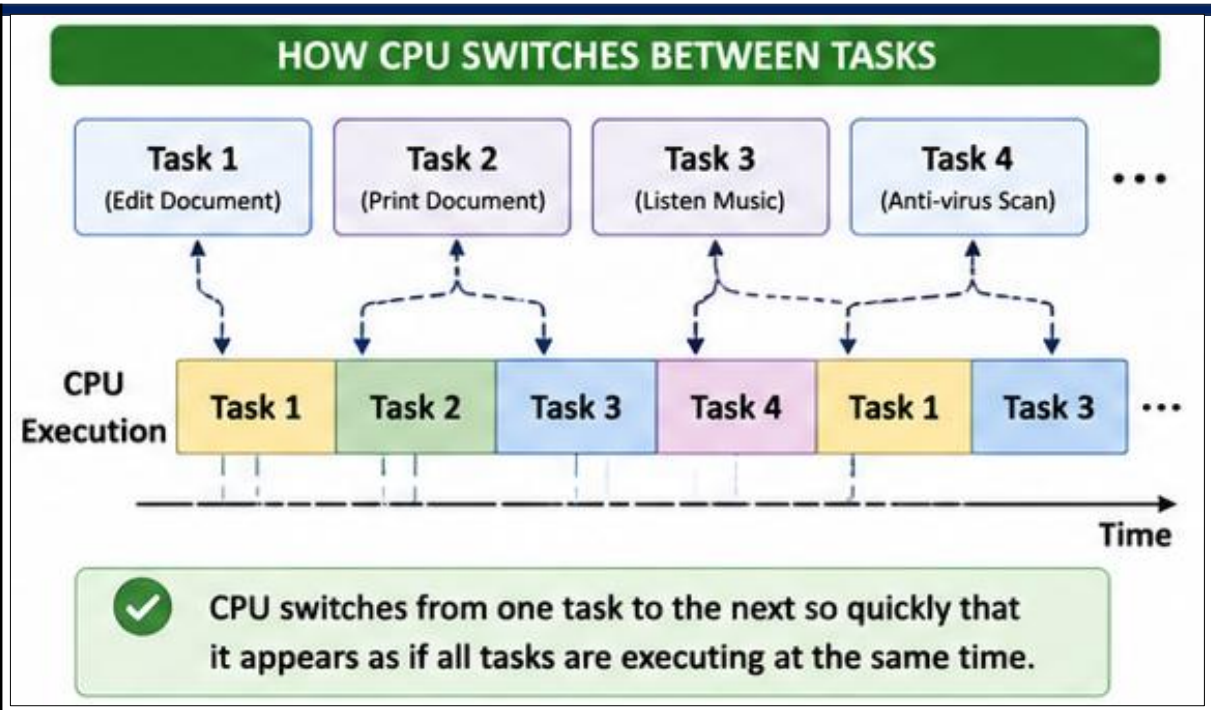
REAL-LIFE ANALOGY

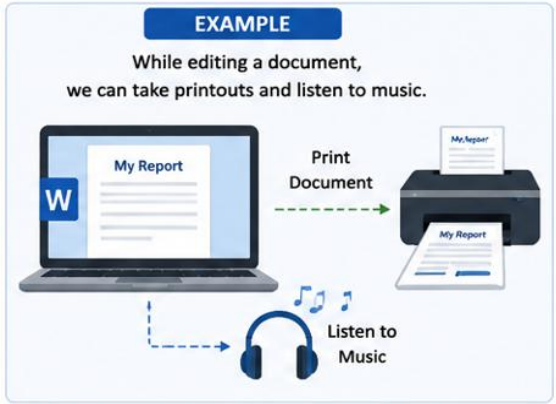
	Recipe Book	→ Program
	Chef cooking from recipe	→ Process
	Chopping, boiling, frying, serving	→ Tasks

In short: **Program** is a passive code. | **Process** is a program in execution. | **Task** is a unit of work within a process.

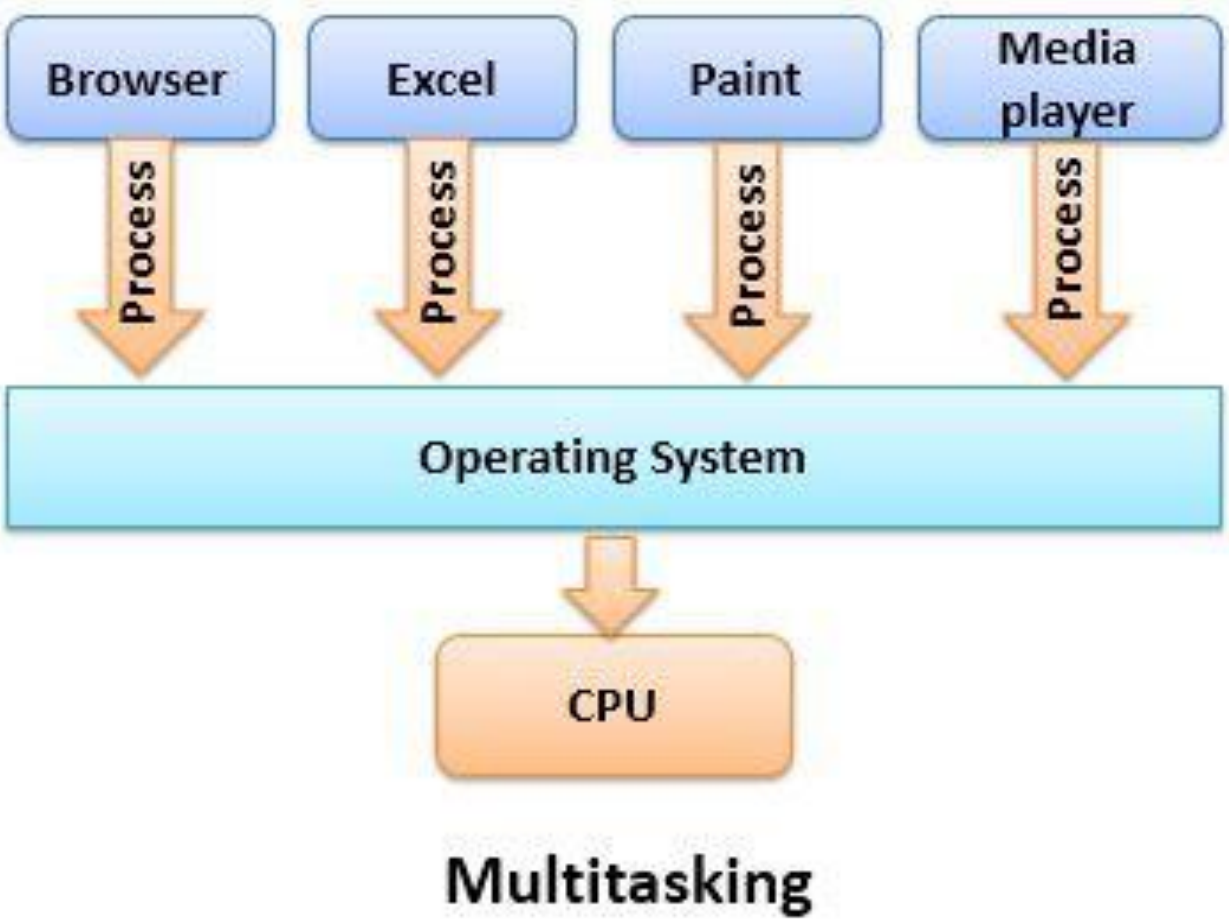
Types of OS: 3. Multitasking Operating System

- Multitasking is the ability of an operating system to execute more than one task simultaneously on a single processor machine
- Multitasking is logical extension of multiprogramming.
- In reality no two tasks on a single-processor machine can be executed at the same time.
- Actually CPU switches from one task to the next task so quickly that appears as if all the tasks are executing at the same time.
- A task is a particular operation like edit, copy, rename, delete, open, close etc.
- Multitasking is possible by executing one job in foreground & many jobs in background.
- Foreground jobs require user interaction where as background jobs does not require user interaction.
- E.g. While editing we can take printouts & listen music.





Types of OS: 3. Multitasking Operating System



Multitasking

HOW MULTITASKING WORKS

1. Multiple tasks are loaded in memory.
2. CPU executes one task for a very short time.
3. CPU quickly switches to the next task.
4. This cycle continues.
5. All tasks appear to run at the same time.

Types of OS: 3. Multitasking Operating System

Advantages :-

- Multiple tasks can be run simultaneously
- Increases sped of execution
- CPU utilization increases.

Disadvantages:-

- These systems are not reliable.
- Problem of data communication.
- It raises question of security of user program & data

Types of OS: Compare Multiprogramming & Multitaking

Sr. No.	Multiprogramming	Multitasking
1	Multiple programs are executed simultaneously.	Multiple tasks are executed simultaneously.
2	Collection of different tasks is program.	Task is smaller unit of program.
3	Program may contain multiple task .	Task do not contain program
4	User interaction is not provided	User interaction is provided
5	It utilizes CPU & I/O devices efficiently	It utilizes CPU efficiently
6	There are simply only programs	There are two types of task foreground & background task.

Types of OS: 2. Multiprogrammed Operating System

- Types: Batch Operating System, Multiprogrammed Operating System, time-sharing operating system, Multiprocessor Operating System, Distributed Operating System, Real-Time Operating System;

Types of OS: 2. Multiprogrammed Operating System

- Types: Batch Operating System, Multiprogrammed Operating System, time-sharing operating system, Multiprocessor Operating System, Distributed Operating System, Real-Time Operating System;

Types of OS: 2. Multiprogrammed Operating System

- Types: Batch Operating System, Multiprogrammed Operating System, time-sharing operating system, Multiprocessor Operating System, Distributed Operating System, Real-Time Operating System;

3. Multitasking Operating System

3. Multitasking Operating System

3. Multitasking Operating System

- Advantages :-

  - Multiple tasks can be run simultaneously
  - Increases sped of execution
  - CPU utilization increases.

- Disadvantages:-

  - These systems are not reliable.
  - Problem of data communication.
  - It raises question of security of user program & data

Multiprogramming Vs Multitasking

Sr. No.	Multiprogramming	Multitasking
1	Multiple programs are executed simultaneously.	Multiple tasks are executed simultaneously.
2	Collection of different tasks is program.	Task is smaller unit of program.
3	Program may contain multiple task .	Task do not contain program
4	User interaction is not provided	User interaction is provided
5	It utilizes CPU & I/O devices efficiently	It utilizes CPU efficiently
6	There are simply only programs	There are two types of task foreground & background task.

4. Time sharing OS

- Time-sharing is a technique which enables many people, located at various terminals, to use a particular computer system at the same time.
- Time-sharing is a logical extension of multiprogramming.
- **Processor's time which is shared among multiple users simultaneously is termed as time-sharing.**
- The main objective of Time-Sharing is to minimize response time.
- Multiple jobs are executed by the CPU by switching between them, but the switches occur so frequently. Thus, the user can receive an immediate response.
- Example OS :Linux

4. Time sharing OS

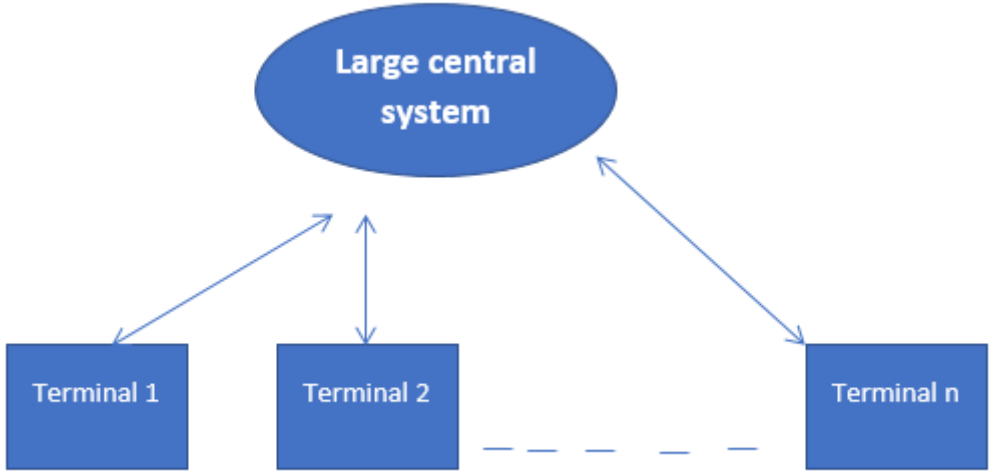


Fig. Time shared OS

4. Time sharing OS

- Advantages :-

- Provides the advantage of quick response
- Avoids duplication of software
- Reduces CPU idle time

- Disadvantages :-

- Expensive to build.
- Problem of data communication
- Difficult & complicated.

5. Multiprocessor OS/Tightly coupled/Parallel System

- In multiprocessor system, two or more processors work together.
- In this system, multiple programs (more than one program) are executed on different processors at the same time. This type of processing is known as multiprocessing.
- Some operating systems have features of multiprocessing. UNIX is an example of multiprocessing operating system. Some versions of Microsoft Windows also support multiprocessing.
- In this more than one processors can be places in single cabinet which shares resources, bus, memory etc.
- **Objective –**is to get maximum work done in less time.
- It is also called as tightly coupled or parallel system.
- The multiprocessor systems are further divided into two types

  (i)Asymmetric multiprocessing system

  (ii)Symmetric multiprocessing system

5. Multiprocessor OS/Tightly coupled/Parallel System

- Types of Multiprocessor System are

1. Symmetric Multiprocessing –

- The multiprocessing system, in which multiple processors work together on the same task, is known as Symmetric Multiprocessing System.
- In this system, each processor can perform all types of tasks.
- All processors are treated equally and no master-slave relationship exists between the processors.
- In this two or more processors are connected with high bandwidth link & manage by single OS. All processor equally share I/O device.

5. Multiprocessor OS/Tightly coupled/Parallel System

2. Asymmetric Multiprocessing –

- Each processor is assigned specific task. In this Master-Slave concept is used. Master processors controls slave processors.
- The multiprocessing system, in which each processor is assigned a specific task, is known as Asymmetric Multiprocessing System.
- For example, one processor is dedicated for handling user's requests, one processor is dedicated for running application program, and one processor is dedicated for running image processing and so on.
- In this system, one processor works as master processor, while other processors work as slave processors.
- The master processor controls the operations of system. It also schedules and distributes tasks among the slave processors. The slave processors perform the predefined tasks.

5. Multiprocessor OS/Tightly coupled/Parallel System

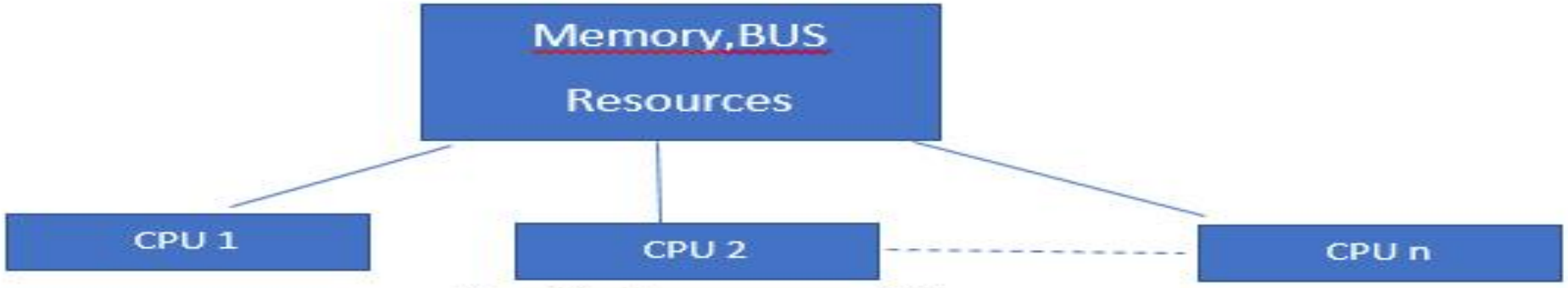


Fig. Multiprocessor OS

OR



Fig. :-Multiprocessing System

5. Multiprocessor OS/Tightly coupled/Parallel System

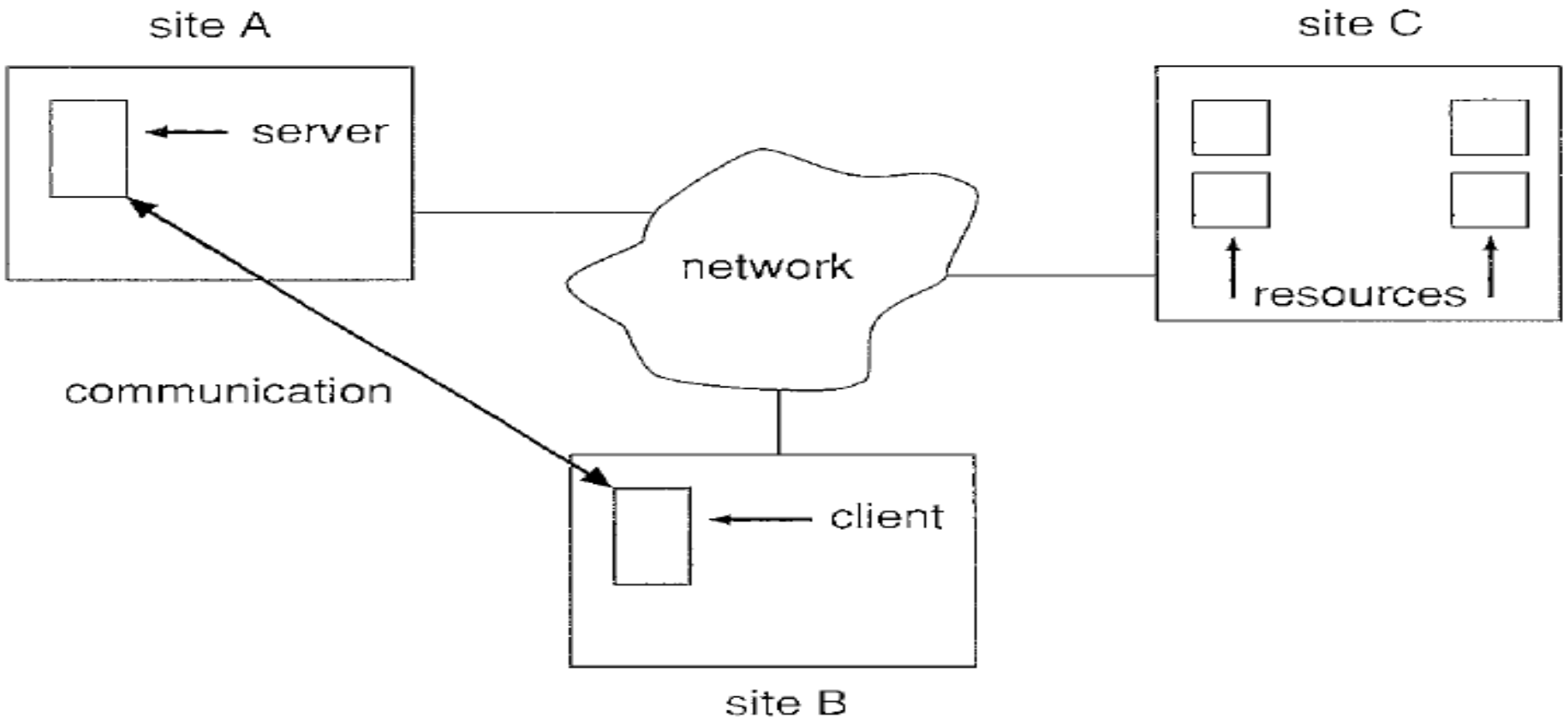
- **Advantages:-**
- It increases throughput.
- It increases reliability (if one processor fails, whole system will not fail.)
- More than one processors can be used.
- Maximum work done can be done in less time.

- **Disadvantages:-**
- Expensive to build & maintain.
- More complex in both hardware & software as compare to uniprocessor system.

6. Distributed Operating System/Loosely coupled System

- It is collection of processors interconnected by communication network.
- Also called as **loosely coupled system** and are exactly opposite to multiprocessor system.
- A Distributed system is basically a collection of autonomous(Independent) Computer systems which co-operate with one another through their hardware & software interconnections.
- The processors communicate with one another through various communication lines (such as high-speed buses or telephone lines).
- In Distributed system, the processors cannot share memory or time each processor has its own local memory.
- Processors in a distributed system may vary in size and function.
- These processors are referred as sites, nodes, computers, and so on.

6. Distributed Operating System/Loosely coupled System



A distributed system.

6. Distributed Operating System/Loosely coupled System

- **Advantages:-**
- With resource sharing facility, a user at one site may be able to use the resources available at another.
- Speedup the exchange of data with one another via electronic mail.
- If one site fails in a distributed system, the remaining sites can potentially continue operating.
- Better service to the customers.
- Reduction of the load on the host computer.
- Reduction of delays in data processing.

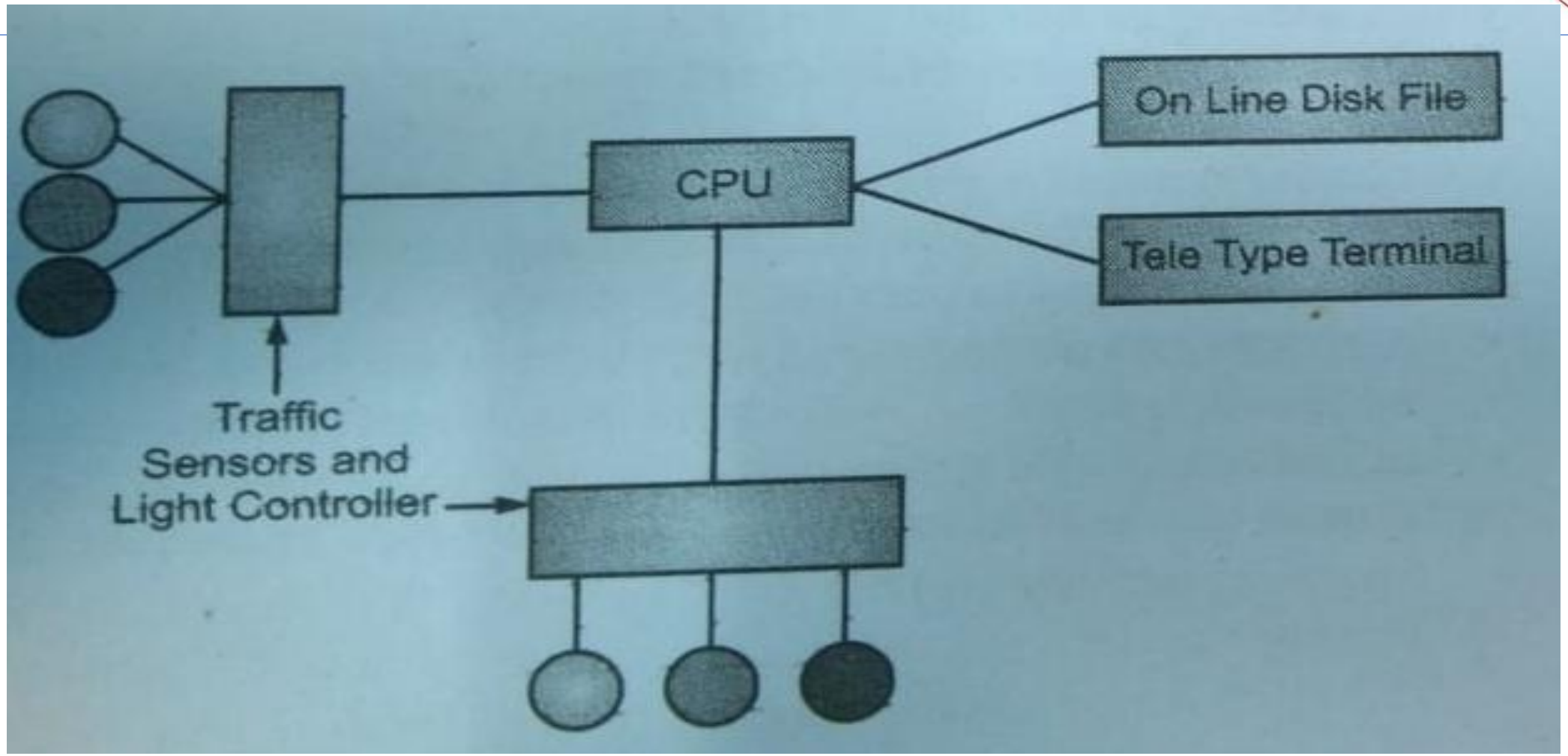
Disadvantages:-

- Complex & difficult to build.
- Special security measures are needed to protect the shared resources.

7. Real time Operating System

- A real time OS is needed for the computer systems controlling a process or real time situation such as a machine or a satellite.
- **Objective** -To provide quick event response time & meet scheduled deadline.
- It is used when rigid time requirement have been placed on operation of a process.
- In this sensors are used, sensors bring data to computer, computer then analyses data & adjust controls according to sensors input.
- The real time OS have to work strict time limits & have to be quick. Apart from this, these systems must be highly reliable to avoid failure of the system being controlled.
- It has fixed time constraints otherwise system will fail.
- Examples: Nuclear power plant control, Railway & airline reservation system, industrial control system, satellite communication, traffic light control etc.

7. **Real time** Operating **System**



7. Real time Operating System

- There are two types of real-time systems:

1. Hard real-time systems

- Hard real-time systems guarantees that critical tasks complete on given time.
- e.g. car engine control system.- delay in particular signal may cause engine failure.

2. Soft real-time systems

Soft real time systems will tolerate delay in time.

E.g. Live audio, live video

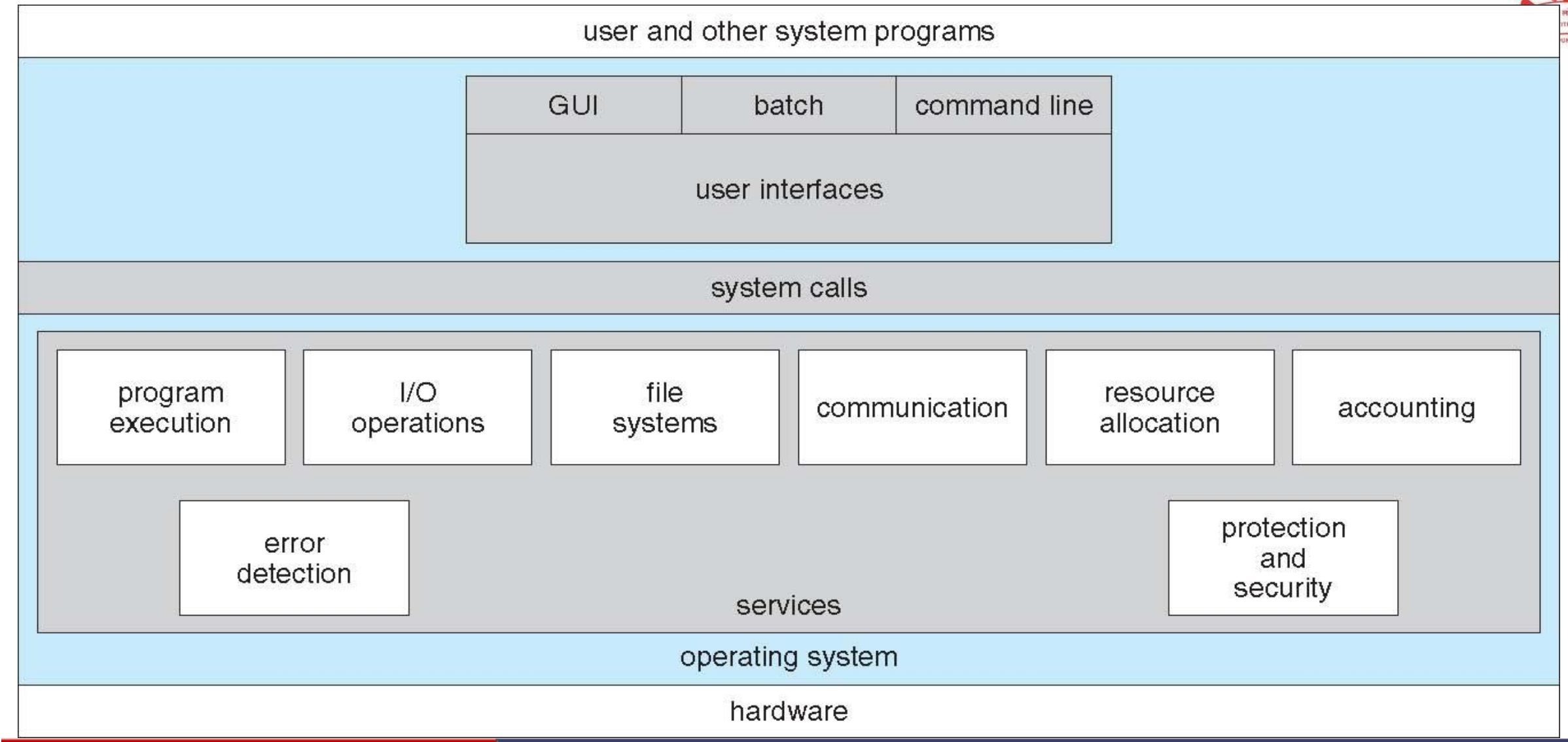
Advantages :-

- Direct communication between user & system is possible
- System generated task can be completed within time

Disadvantages :-

More complex & difficult to maintain

Operating System Services



Operating System Services- Provided to User

1. User Interface
2. Program execution
3. I/O operations
4. File system manipulation
5. Communication
6. Error detection

Operating System Services- Provided to User

- **User interface** - Almost all operating systems have a user interface (UI).
  - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**
- **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
- **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
- **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

Operating System Services- Provided to User

- **Communications** – Processes of exchange information, on the same computer or between computers over a network
- Communications may be via shared memory or through message passing (packets moved by the OS)
- **Error detection** – OS needs to be constantly aware of possible errors
  - May occur in the CPU and memory hardware, in I/O devices, in user program
  - For each type of error, OS should take the appropriate action to ensure correct and consistent computing.
  - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system.

Operating System Services- Provided to OS itself

1.Resource Allocation

2. Accounting

3. Protection & Security

Operating System Services- Provided to OS itself

- **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
  - Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
- **Accounting -** To keep track of which users use how much and what kinds of computer resources
- **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
  - **Protection** involves ensuring that all access to system resources is controlled
  - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
  - If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.

System Calls

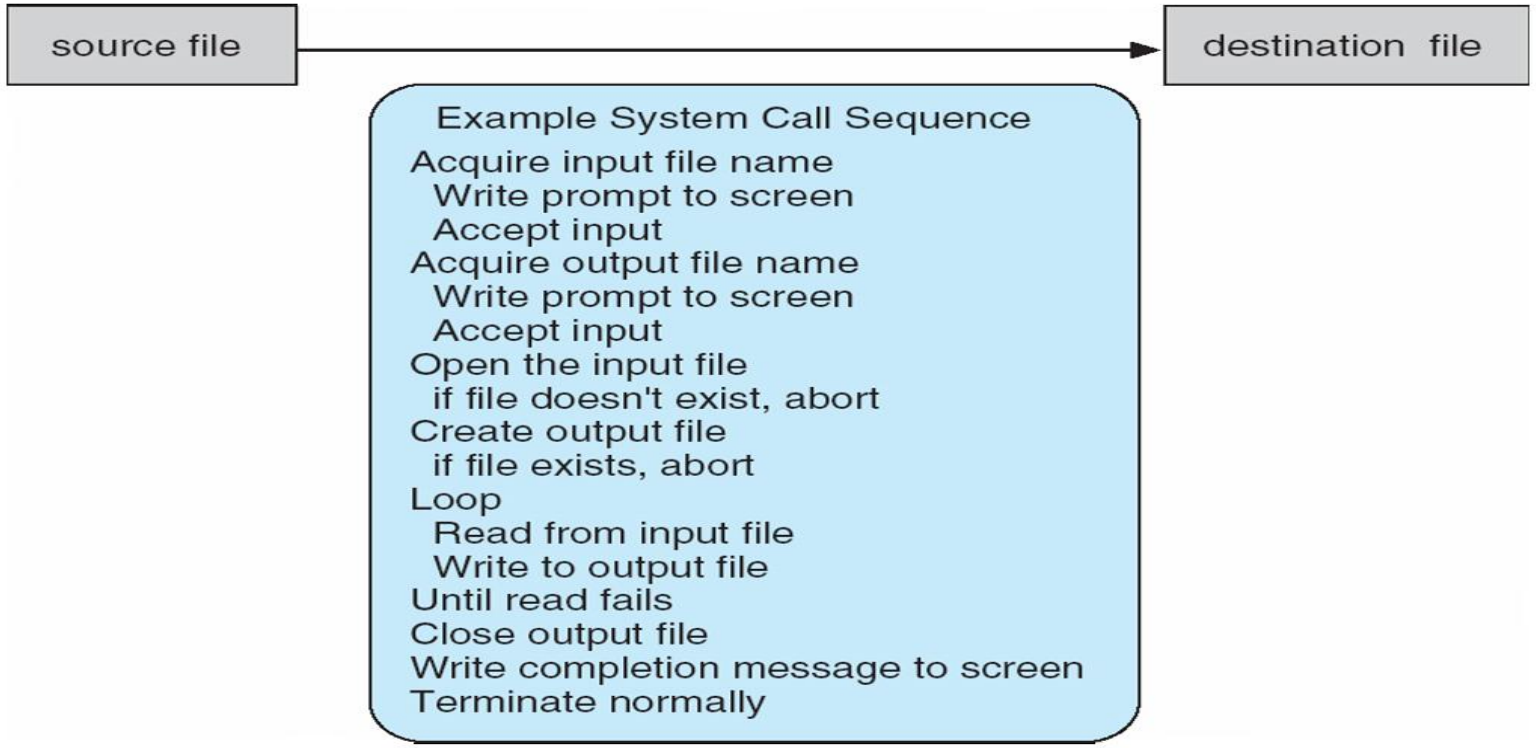
- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)

- Mostly accessed by programs via a high-level **Application Program Interface (API)** rather than direct system call use

- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)
- Why use APIs rather than system calls?

Example of System Calls

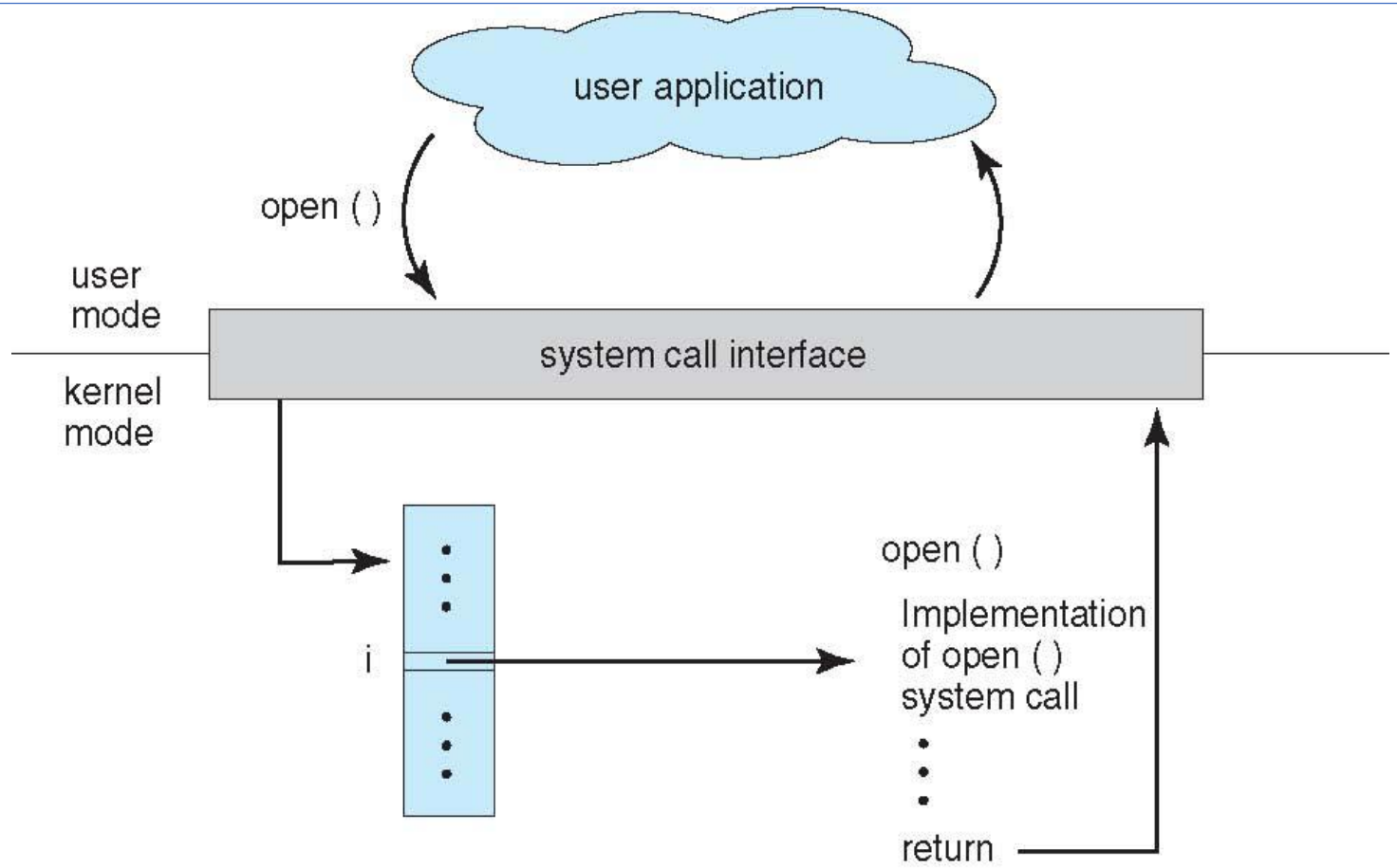
System call sequence to copy the contents of one file to another file



System Call Implementation

- Typically, a number associated with each system call
  - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

API – System Call – OS Relationship



Types of System Calls

- **Process control**
  - end, abort
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
- **File management**
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes

Types of System Calls (Cont.)

- **Device management**
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices
- **Information maintenance**
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- **Communications**
  - create, delete communication connection
  - send, receive messages
  - transfer status information
  - attach and detach remote devices

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

THANK YOU